

Self-Adaptive Smart Parking Lot Manager

Course: Software Engineering for Autonomous Systems

Team Members: Jawayria Hashmi, Lameya Islam

1. Introduction

Traditional parking management requires constant human oversight and manual adjustments. Modern parking systems face challenges such as inefficient space utilization, queue buildup and congestion during peak hours, revenue loss due to rejected vehicles, and limited real-time visibility. An autonomous, self-adaptive system can address these challenges by reacting to changing conditions automatically, optimizing multiple competing objectives and adapting strategies without human intervention.

This project presents a **Self-Adaptive Smart Parking Lot Manager**, an autonomous system that dynamically regulates parking operations using the **MAPE-K (Monitor-Analyze-Plan-Execute-Knowledge)** autonomic control loop. The system continuously monitors parking lot conditions through simulated sensors and autonomously adapts its behavior to optimize utilization, minimize congestion, and maximize revenue without requiring human intervention.

The project simulates a multi-lot parking system with:

- 2 parking lots (Lot 1: 100 spaces, Lot 2: 50 spaces)
- Dynamic pricing (\$1-\$20 range)
- Automated gate control (open/close based on queue length and occupancy) Real-time monitoring dashboard

2. System Goals and Objectives

2.1. Primary Goals

#	Description	Metric	Target
1	Maintain Optimal Utilization	Occupancy %	70-95%
2	Minimize Congestion	Queue Length	< 8 vehicles
3	Reduce Rejections	Rejected Count	Minimize
4	Maximize Revenue	Total Revenue	Maximize

2.2. Adaptation Objectives

1. **Dynamic Pricing:** Adjust prices based on occupancy levels
 - Increase price when occupancy is more than high_occupancy_threshold (discourage arrivals)
 - Decrease price when occupancy is less than low_occupancy_threshold (attract customers)
2. **Gate Control:** Manage vehicle flow automatically
 - Close gate when occupancy is greater than or equal to gate_close_occupancy_threshold (prevent overflow)
 - Reopen gate when occupancy drops below gate_reopen_occupancy_threshold.

3. Architecture Design

3.1. High Level Architecture

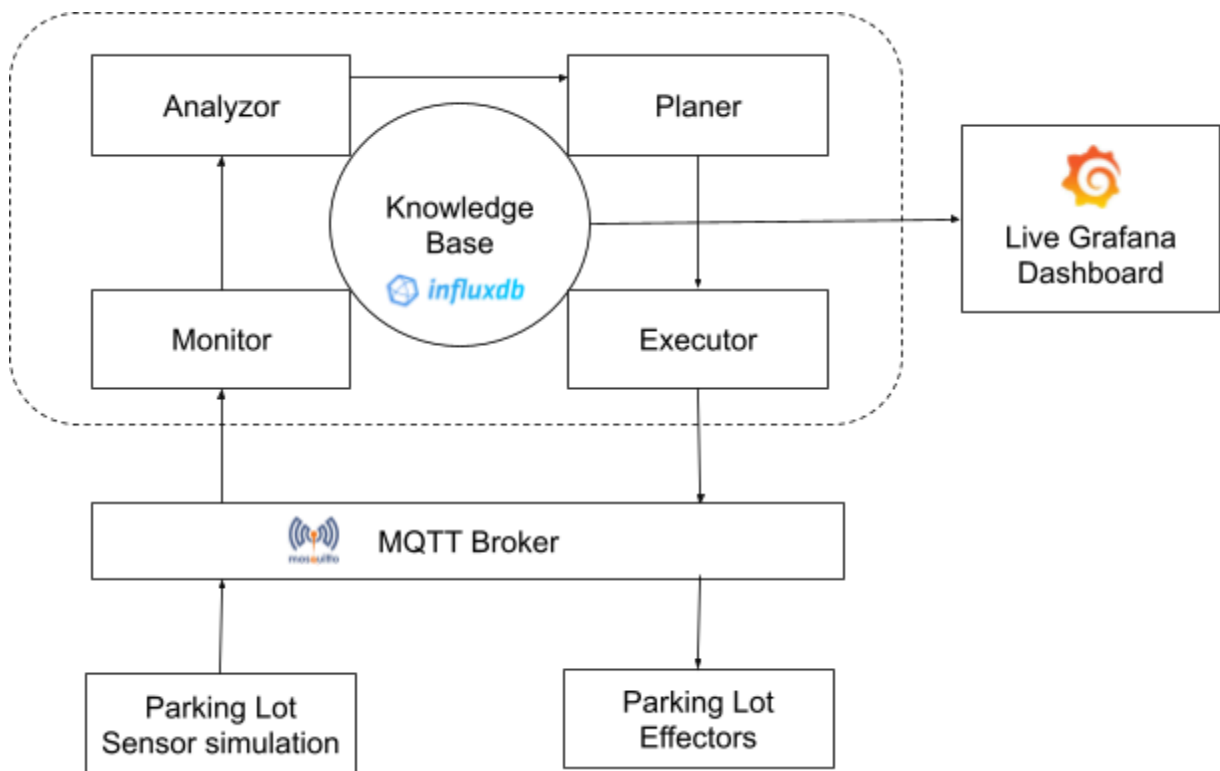


Figure: High Level Architecture Diagram for Self Adaptive Smart Parking Lot Manager

3.2. Component Description

Component	Technology	Purpose
-----------	------------	---------

Autonomic Manager	Python	MAPE-K control loop implementation
Message Broker	MQTT (Mosquitto)	Asynchronous communication
Knowledge Base	InfluxDB	Persistent storage of system state and history
Dashboard	Grafana	Real-time visualization
Parking Simulator	Python	Simulates parking lot behavior

All services are deployed using Docker Compose to ensure reproducibility and isolation. The system three containerized services:

- MQTT Broker: Eclipse Mosquitto on port 1883
- InfluxDB: Time-series database on port 8086
- Grafana: Visualization dashboard on port 3000

3.3. Communication Architecture

The system uses MQTT publish/subscribe messaging for loose coupling between components. The following topics are used:

Topic Pattern	Direction	Purpose
parking/lot/{lot_id}/sensors	Lot → Manager	Sensor data
parking/lot/{lot_id}/control	Manager → Lot	Control commands
parking/lot/{lot_id}/status	Lot → Manager / Storage	Status updates
parking/system/metrics	Manager → All	System-wide metrics
parking/system/adaptations	Manager → Log	Adaptation events

Message Flow:

1. Parking lot simulators publish sensor data every 2 seconds.
2. Autonomic manager subscribes to sensor data.
3. The manager analyzes data and makes decisions.
4. The manager publishes control commands.
5. Lots receive and execute commands.
6. All data stored in InfluxDB for persistence.
7. Grafana queries InfluxDB for visualization.

4. MAPE-K Loop Implementation

4.1. Monitor Component

Purpose: Collect real-time sensor data from all parking lots

The Monitor component subscribes to MQTT topics for each parking lot and collects the data like occupancy, queue length, rejected count, gate state etc. at 2-second intervals. Upon receiving sensor data, the Monitor stores it in the Knowledge Base and triggers a MAPE-K analysis cycle.

4.2. Analyze Component

Purpose: Detect deviations from system goals and assess severity

The Analyze component evaluates the current state against predefined thresholds and classifies issues by severity. The analysis produces an AnalysisResult containing a list of detected issues, severity classification, current system state and recommended adaptation actions.

4.3. Plan Component

Purpose: Select appropriate adaptation actions based on analysis

The Plan component uses a rule-based decision matrix to select actions. A 10-second cooldown period between adaptations per lot prevents oscillation and ensures system stability. The Plan component creates an AdaptationDecision containing selected actions (limited to 2 per cycle to prevent conflicting or excessive adaptations), trigger conditions, expected outcome prediction and confidence level.

4.4. Execute Component

Purpose: Apply planned adaptations via MQTT commands

The Execute component translates adaptation decisions into control commands and publishes them to the appropriate MQTT topics. Each command includes: - Target lot identifier - Action type (INCREASE_PRICE, CLOSE_GATE, etc.) - Action parameters (new price) - Timestamp and reason for audit trail

After execution, the decision is stored in the Knowledge Base for historical analysis and the cooldown timer is reset.

4.5. Knowledge Component

Purpose: Persistent storage of system state, history, and rules

The Knowledge Base uses InfluxDB to store parking sensor data, adaptation decisions and system metrics. The Knowledge Base provides query interfaces for retrieving current state for analysis, computing trends and averages and accessing adaptation history for learning. It also supports retrospective analysis through historical queries.

5. Adaptation Strategies

5.1. Dynamic Pricing Strategy

Objective: Balance demand through price elasticity

The dynamic pricing strategy adjusts parking rates based on current occupancy levels:

- **High Occupancy:** Increase price by \$1.00 per adaptation cycle to discourage new arrivals
- **Low Occupancy:** Decrease price by \$0.50 per adaptation cycle to attract customers
- **Normal Range:** Maintain current price

5.2. Gate Control Strategy

Objective: Prevent queue buildup and system overload

The gate control strategy manages vehicle flow through automatic gate operation:

Close Gate: $\text{Occupancy} \geq \text{gate_close_occupancy_threshold}$

Open Gate: $\text{Occupancy} < \text{gate_reopen_occupancy_threshold}$

When the gate is closed, new arrivals join the queue (up to maximum 15 vehicles), departures continue normally, reducing occupancy, after which queued vehicles enter as spaces become available.

The dual-condition requirement for gate reopening prevents premature opening when the queue is still high, which was identified as a critical issue during testing.

6. Conclusion

This project successfully implements a **Self-Adaptive Smart Parking Lot Manager** using the MAPE-K autonomic computing reference architecture. The system demonstrates autonomous operation, goal-directed behavior and observability. The project also highlights the importance of stability mechanisms such as cooldown periods and multi-condition triggers to avoid oscillatory behavior. Overall, the system demonstrates how autonomic control loops can effectively manage complex, dynamic environments through well-defined goals and feedback-driven adaptation.

7. Limitations and Future Work

The current system is rule-based and does not learn from past adaptations. Future work could extend the Knowledge component with learning mechanisms to adjust thresholds dynamically. Additionally, real-world deployment would require integration with physical sensors and actuators, as well as robustness against sensor noise and communication delays.